N-ary Proof Search Trees as a Directed Container

Extracted from a machine-checked development

Abstract

We generalise binary proof search trees to finitely branching rose trees of arbitrary arity and exhibit them as a directed container. Shapes are rose trees; positions are paths from the root indexed by finite ordinals; the directed structure is path concatenation. The five directed-container laws hold by structural recursion on paths. The induced comonad extracts the root label and duplicates by replacing each label with the full arbitrarily branching subsearch from that node. A coKleisli morphism is a proof strategy that may inspect the entire search tree. All stated results have been machine-checked.

Contents

1 Introduction **1**

2 Shapes and positions **2**

3 Subtree and path concatenation **2**

4 The directed-container laws **2**

5 The induced comonad **3**

6 Relation to binary search trees **3**

7 Discussion **3**

1 Introduction

Proof search need not be binary: a tactic may offer any finite number of alternatives, and dead ends or successes may occur at any arity. Rose trees—nodes with a finite list of children—are the natural shapes. We encode branching by a natural number n together with a family of children indexed by the finite ordinal $\mathrm{Fin}(n)$, so that structural recursion works directly on trees and paths without auxiliary well-founded arguments.

The directed-container structure is identical in spirit to the binary case: the root is the empty path; subtrees are obtained by descending along a path; shift is path concatenation. The five laws hold by the same style of structural recursion.

What this note claims. In ordinary mathematical language: finitely branching rose trees with path positions form a directed container. Binary search trees embed into this structure (leaves as arity 0, binary nodes as arity 2), but the N-ary container is not a strict specialisation of the binary one, since every natural number is allowed as branching factor. The induced comonad navigates arbitrarily branching search spaces while preserving dead ends at every depth.

2 Shapes and positions

Definition 2.1 (Rose tree)**.** A **finitely branching rose tree** is an element of the inductive type RTree generated by a single constructor

$$\mathrm{node} \;:\; (n : \mathbb{N}) \to \big(\mathrm{Fin}(n) \to \mathrm{RTree}\big) \to \mathrm{RTree}. \tag{1}$$

A leaf (dead end or success) is $\mathrm{node}(0, f)$ where $f : \mathrm{Fin}(0) \to \mathrm{RTree}$ is vacuous. The only position in a leaf is here.

Remark 2.2 (Encoding choice)**.** An encoding via lists of children is more familiar but introduces a nested inductive occurrence (List wraps the recursive argument). The $\mathrm{Fin}(n)$-family encoding avoids that complication and lets structural recursion work directly on RTree and RPos.

Definition 2.3 (Position)**.** A **position** (path) in a rose tree t is an element of the inductive family $\mathrm{RPos}(t)$ generated by

- here — the root of t;
- $\mathsf{child}(i, p)$ — for $t = \mathrm{node}(n, c)$ and $i \in \mathrm{Fin}(n)$, descend into the i-th child $c(i)$ and continue along path p.

For a leaf ($n = 0$), there is no valid $i \in \mathrm{Fin}(0)$, so the only position is here.

3 Subtree and path concatenation

Definition 3.1 (Subtree)**.** The **subtree** operation $\downarrow : (t : \mathrm{RTree}) \to \mathrm{RPos}(t) \to \mathrm{RTree}$ is defined by

$$t \downarrow \mathsf{here} \;=\; t, \tag{2}$$

$$\mathrm{node}(n, c) \downarrow \mathsf{child}(i, p) \;=\; c(i) \downarrow p. \tag{3}$$

Definition 3.2 (Shift)**.** The **shift** (path concatenation)

$$\oplus \;:\; (t : \mathrm{RTree}) \to (p : \mathrm{RPos}(t)) \to \mathrm{RPos}\big(t \downarrow p\big) \to \mathrm{RPos}(t) \tag{4}$$

is defined by

$$\mathsf{here} \oplus q \;=\; q, \tag{5}$$

$$\mathsf{child}(i, p) \oplus q \;=\; \mathsf{child}\big(i,\; p \oplus q\big). \tag{6}$$

4 The directed-container laws

As in the binary case, a directed container is a container with root, subtree, and shift operations satisfying laws D1–D5. We take Shape = RTree, Pos = RPos, $o(t) = \mathsf{here}$, and the operations of Definitions 3.1 and 3.2.

Proposition 4.1 (D1 and D2, definitional)**.** *For every rose tree t and every position q in t,*

$$t \downarrow \mathsf{here} \;=\; t, \qquad \mathsf{here} \oplus q \;=\; q. \tag{7}$$

Proof. By lean. □

Proposition 4.2 (D3: right identity)**.** *For every rose tree t and every position p in t,*

$$p \oplus \mathsf{here} \;=\; p. \tag{8}$$

Proof. By lean. □

Proposition 4.3 (D4: nesting of subtrees)**.** *For every rose tree t, position p in t, and position q in the subtree at p,*

$$t \downarrow (p \oplus q) \;=\; (t \downarrow p) \downarrow q. \tag{9}$$

Proof. By lean. □

Proposition 4.4 (D5: associativity of shift)**.** *For every rose tree t and compatible positions p, q, r,*

$$(p \oplus q) \oplus r' \;=\; p \oplus (q \oplus r), \tag{10}$$

where r' is r transported along the equality of fibres supplied by D4.

Proof. By lean. □

Theorem 4.5 (N-ary proof search as directed container)**.** *The data*

$$(\mathrm{RTree},\ \mathrm{RPos},\ o = \mathsf{here},\ \downarrow,\ \oplus)$$

form a directed container: laws D1–D5 hold.

Proof. By lean. □

5 The induced comonad

Remark 5.1 (Extract and duplicate)**.** The directed container induces a comonad on the extension. For a labelling $v : \mathrm{RPos}(t) \to X$,

$$\mathrm{extract}\,\langle t, v\rangle \;=\; v(\mathsf{here}), \tag{11}$$

$$\mathrm{duplicate}\,\langle t, v\rangle \;=\; \Big\langle t,\ p \mapsto \langle t \downarrow p,\ q \mapsto v(p \oplus q)\rangle \Big\rangle. \tag{12}$$

Thus extract reads the label at the root, and duplicate replaces the label at every node by the full sub-search from that node, at arbitrary branching factor.

Remark 5.2 (Proof strategies)**.** A coKleisli morphism $W(X) \to Y$ is a proof strategy that may inspect the entire (arbitrarily branching) search tree to decide what to produce.

6 Relation to binary search trees

Remark 6.1 (Embedding)**.** Binary search tree shapes embed into rose trees: a leaf maps to $\mathrm{node}(0, !)$, and a binary node $\mathsf{node}(\ell, r)$ maps to $\mathrm{node}\big(2,\ [\![\ell', r']\!]\big)$, where ℓ' and r' are the recursive images. This is not a strict sub-case of the binary theory: binary trees force arity 0 or 2 at every node, while rose trees allow any natural number as branching factor.

7 Discussion

What has been established. Finitely branching rose trees with path positions form a directed container. The five laws hold by structural recursion on paths; D1 and D2 hold definitionally. The induced comonad navigates arbitrarily branching search spaces while preserving dead ends at every depth.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant.

Acknowledgements

The mathematical content follows the theory of directed containers of Ahman, Chapman, and Uustalu, generalising the binary construction. The presentation here is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] D. Ahman, J. Chapman, and T. Uustalu. When is a container a comonad? In *FoSSaCS*, volume 7213 of *LNCS*, pages 74–88. Springer, 2012.

[2] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.